

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	J.Raju	Reg. No.:	192325117
Section / Batch:	Slot-A	Date:	

Code of Conduct

I J.Raju (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J.Raju

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Aim

To write a Python program that demonstrates the use of regular expressions to match and search for patterns in text.

Algorithm

1. Start the program.
2. Import the re module.
3. Store a sample text in a variable.
4. Define a regular expression pattern for an email address.
5. Use re.search() to find the email in the text.
6. Define a regular expression pattern for a phone number.
7. Use re.search() to find the phone number.
8. Display the matched email and phone number.
9. Stop the program.

Python Code

```
import re
text = "My email is student123@gmail.com and my phone number is 9876543210."
email_pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b'
email_match = re.search(email_pattern, text)
if email_match:
    print("Email found:", email_match.group())
phone_pattern = r'\b\d{10}\b'
phone_match = re.search(phone_pattern, text)
if phone_match:
    print("Phone number found:", phone_match.group())
word_pattern = r'\w*\d+\w*'
words = re.findall(word_pattern, text)
print("Words containing digits:", words)
```

Output

```
.. Email found: student123@gmail.com
   Phone number found: 9876543210
   Words containing digits: ['student123', '9876543210']
```

Result

Thus, the Python program to demonstrate the use of regular expressions for matching and searching patterns in text was successfully executed and the output was verified.

2.Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Aim

To implement a basic Finite State Automaton (FSA) in Python that recognizes strings ending with "ab".

Algorithm

1. Start the program.
2. Define the states of the automaton.
3. Read the input string.
4. Process each character one by one and change the state.
5. After processing all characters, check the final state.
6. If the final state is the accepting state, display "Accepted".
7. Otherwise, display "Rejected".
8. Stop the program.

Python Code

```
def fsa_ends_with_ab(string):  
    state = 0
```

```
    for ch in string:
```

```
        if state == 0:
```

```
            if ch == 'a':
```

```
                state = 1
```

```
            else:
```

```
                state = 0
```

```
        elif state == 1:
```

```
            if ch == 'b':
```

```
                state = 2
```

```
            elif ch == 'a':
```

```
                state = 1
```

```
            else:
```

```
                state = 0
```

```
        elif state == 2:
```

```
            if ch == 'a':
```

```
                state = 1
```

```
            else:
```

```
                state = 0
```

```
    if state == 2:
```

```
        print("Accepted")
```

```
    else:
```

```
        print("Rejected")
```

```
text = input("Enter a string: ")  
fsa_ends_with_ab(text)
```

```
Enter a string: cab  
Accepted
```

```
• Enter a string: baaa  
Rejected
```

Result

Thus, the Python program to implement a Finite State Automaton (FSA) that recognizes strings ending with "ab" was successfully executed and the output was verified.

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Aim

To write a Python program that demonstrates morphological analysis using the **NLTK** library by performing stemming and lemmatization.

Algorithm

1. Start the program.
2. Import the required NLTK modules.
3. Create a list of sample words.
4. Initialize the **Porter Stemmer** and **WordNet Lemmatizer**.
5. Apply stemming to each word.
6. Apply lemmatization to each word.
7. Display the original word, stemmed word, and lemmatized word.
8. Stop the program.

Python Code

```
import nltk

from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('wordnet')

words = ["running", "played", "better", "cars", "studies"]

stemmer = PorterStemmer()

lemmatizer = WordNetLemmatizer()

print("Word\t\tStem\t\tLemma")

for word in words:

    stem = stemmer.stem(word)

    lemma = lemmatizer.lemmatize(word)

    print(word, "\t", stem, "\t\t", lemma)
```

Sample Output

```
... Word          Stem          Lemma
    running        run          running
    played   play    played
    better   better
    cars     car     car
    studies      studi    study
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Result

Thus, the Python program to perform **morphological analysis** using the **NLTK** library was successfully executed using **stemming** and **lemmatization**, and the output was verified